Lecture 9-10: Virtual Memory

Emergency Crash Study Guide for Operating Systems - CS x61

How to use this guide in the next few hours: First read the one-page map and the bold exam traps. Then study page faults, address translation, page tables/TLB, replacement algorithms, and resident set/thrashing policies. Finish by testing yourself with the likely questions and flashcards.

0. The 90-minute emergency plan

Time	What to study	Why it matters
0-15 min	Virtual memory idea, demand paging, valid-invalid bit, page fault steps	These are definition/sequence questions. Easy marks.
15-35 min	Address translation: page number + offset, PTE bits, TLB hit/miss	Likely diagram/calculation questions.
35-55 min	Page table structures: hierarchical, hashed, inverted; segmentation; combined paging/segmentation	Common comparison questions.
55-75 min	Page replacement: OPT, FIFO, LRU, Clock, modified Clock	Likely algorithm tracing / page fault count questions.
75-90 min	Resident set, working set, PFF, cleaning policy, load control, program structure example	Professor edge cases and explanation questions.

The whole lecture in one sentence: Virtual memory lets a process behave as if it has a large contiguous logical memory, while the OS and hardware keep only needed pages/segments in physical memory and move the rest to disk.

1. One-page mental map

- **Virtual memory:** Separates logical memory seen by the user/program from physical memory. The logical address space can be larger than real RAM.
- **Demand paging:** Pages are loaded only when referenced. If the page is not in memory, the hardware triggers a page fault.
- **Page fault:** Trap to OS -> validate reference -> find frame -> read page from disk -> update page table -> restart instruction.
- **Resident set:** The part of a process currently in main memory.
- **Locality:** Programs tend to reference a small cluster of pages for a period of time. VM works because of locality.
- **Thrashing:** Too many page faults; CPU spends most time swapping pages instead of executing instructions.
- **Page table:** Maps virtual page numbers to physical frame numbers or disk locations. Each process has its own page table.
- **TLB:** A fast cache of recently used page table entries, used to avoid two physical memory accesses for every virtual memory reference.
- **Replacement policy:** When memory is full, choose a victim page to remove. Good policy minimizes future page faults.
- **Resident set management:** Decides how many frames each process gets and whether replacement is local or global.

2. Virtual Memory basics

- Code must be in memory to execute, but most programs do not use all code/data at once. Error routines, unusual branches, and huge data structures may not be needed now.
- VM means the OS does not need to load every piece of a process into main memory.
- A process may be larger than physical memory.
- More processes can be kept ready because each process occupies less RAM while running.
- Less I/O is needed at startup because only needed pages are brought in.

Exam wording: Virtual address space is the logical view of the process. It usually starts at address 0 and is contiguous. Physical memory is organized into page frames. The MMU maps logical/virtual addresses to physical addresses.

Memory hierarchy idea

Level	Transfer unit	What illusion it gives
Registers	Words via load/store	Fastest direct CPU storage
Cache	Lines automatically on cache miss	Illusion of very high speed
Main memory	Pages/frames interact with VM	Reasonable cost, but slower and smaller
Virtual memory / disk	Pages automatically on page fault	Illusion of very large memory

3. Demand Paging and valid-invalid bit

- **Demand paging:** Load a page only when it is demanded during execution.
- **Swapper:** Moves an entire process between memory and disk.
- **Pager:** Moves individual pages between memory and disk.
- **Valid bit / present bit:** Page is in memory.
- **Invalid bit:** Page is not currently in memory, or the reference is illegal depending on context.

Do not confuse: Invalid in the page table can mean either not resident or illegal. The OS checks the PCB/process information to decide whether the reference is valid but not resident, or truly invalid.

When a page is referenced

1. CPU generates a virtual/logical address.
2. MMU uses the page number to look in the page table.
3. If the page is present/valid: continue and form the physical address.
4. If the page is not present/invalid: trap to the OS as a page fault.

Page fault handling sequence

1. OS checks the PCB / process information to see whether the reference is valid.
2. If invalid: terminate the process. If valid: bring the page into memory.
3. Find a free frame in physical memory.
4. Schedule disk I/O to read the missing page into that frame.
5. When disk read completes, update the page table: frame number + valid/present bit.
6. Restart the instruction that caused the trap.

Professor trap: The instruction is restarted, not skipped. The process was interrupted before completing the memory reference.

Execution with VM

- **Resident set:** The subset of a process currently in physical memory.
- The process runs as long as all referenced pages/segments are in its resident set.

- On a page fault, the OS blocks that process, starts disk I/O, dispatches another process, and later puts the original process back in the Ready Queue when I/O finishes.

4. Locality and thrashing

- **Principle of locality:** Program and data references tend to cluster. For a short period, only a small set of pages is heavily used.
- VM depends on locality: keeping the active cluster in memory avoids many page faults.
- **Thrashing:** The OS repeatedly throws out pages just before they are needed again, so the CPU spends most of its time swapping rather than executing.

How to explain thrashing in an exam: Thrashing happens when the resident sets are too small or the multiprogramming level is too high. Page faults explode, disk I/O dominates, and CPU utilization collapses.

5. Paging and address translation

- Each process has its own page table.
- Each page table entry maps a virtual page to a physical frame, or contains the disk address if the page is not resident.
- A virtual address is split into: page number + page offset.
- A physical address is split into: frame number + same offset.
- The page offset does not change during translation.

Address translation formula

Let page size P = 2^p bytes.

Let virtual address size = n bits.

Let physical address size = m bits.

Virtual address = [virtual page number: n-p bits] [offset: p bits]

Physical address = [physical frame number: m-p bits] [offset: p bits]

Offset remains unchanged. Only the page number is translated to a frame number.

Page Table Entry (PTE) contents

PTE field	Meaning / use
Frame number	Physical frame where the page is located if resident.
Present / valid bit (P)	Whether the page is currently in main memory. If absent, reference may cause a page fault.
Modified / dirty bit (M)	Whether page changed since it was loaded. Dirty pages must be written to disk before replacement.
Accessed / use / referenced bit	Shows whether page was recently used; important for Clock and working set approximations.
Protection bits	Read/write/execute permissions. Used for memory protection.
Disk address	Where the page is located in backing store if not in physical memory.

Important: A page fault is expensive because it may require disk I/O, page replacement, page table updates, a process block, and a process switch.

6. Page table structures

- Straight page tables can become huge.
- Example: 32-bit logical address, 4 KB page size -> 2^20 pages -> about 1 million PTEs. If each PTE is 4 bytes, the page table alone is about 4 MB per process.
- The problem is worse for 64-bit address spaces, so page tables must be structured intelligently.

Structure	Core idea	Advantage	Disadvantage / trap
Hierarchical / multi-level	Break the logical address space into multiple page tables. Root table points to second-level tables.	Avoids one huge contiguous page table.	More memory accesses; not great for very large 64-bit spaces if too many levels are required.
Hashed page table	Hash the virtual page number into a table; each bucket has a chain for collisions.	Good for address spaces larger than 32 bits and sparse address spaces.	Must search chain after hash; clustered page tables can map several nearby pages per entry.
Inverted page table	One entry per physical frame, not one entry per virtual page.	Greatly reduces page table memory.	Slower lookup; needs hashing. Entry includes page number, process ID, control bits, chain pointer.

Two-level paging example

- For a 32-bit address and 4 KB pages: offset = 12 bits, virtual page number = 20 bits.
- For a 32-bit address and 1 KB pages: offset = 10 bits, page number = 22 bits.
- If the page table is paged, the page number can be split into p1 and p2. Example: p1 indexes the outer table, p2 indexes the inner table, offset selects the byte inside the page.

7. Translation Lookaside Buffer (TLB)

- Without a TLB, every virtual memory reference can require two physical memory accesses: one to fetch the PTE and one to fetch the actual data/instruction.
- A TLB is a high-speed cache of recently used page table entries.
- TLB hit: frame number is found immediately and physical address is formed.
- TLB miss: processor/page-table hardware checks the page table; if page is not in memory, a page fault occurs; then the TLB is updated.

Exam trap: A TLB miss is not automatically a page fault. A miss only means the translation is not in the TLB. The page may still be in main memory.

8. Segmentation and combined paging/segmentation

- **Segmentation:** Programmer views memory as multiple logical segments, such as code, stack, heap, data, shared library, table, module.
- Segments are unequal and dynamic in size.
- Segmentation supports growing data structures, independent compilation, sharing, and protection.
- Each segment table entry contains the base/start address, length, presence bit, modified bit, and protection information.

Technique	Visible to programmer?	Fragmentation	Strength
Paging	Transparent to programmer	No external fragmentation; small internal fragmentation in last page	Simple fixed-size frames; efficient allocation.
Segmentation	Visible to programmer	May have external fragmentation	Natural modules, sharing, protection, growing structures.

Combined paging + segmentation	Program sees segments; OS pages each segment	Reduces external fragmentation and keeps segmentation benefits	Address translation uses segment table, then page table.

Combined address translation

1. Virtual address contains segment number + offset inside that segment.
2. Processor uses segment number to index the segment table.
3. Segment table points to the page table for that segment.
4. Segment offset is split into page number + page offset.
5. Page number indexes the segment page table and gives frame number.
6. Frame number + page offset forms the physical address.

9. Page size design

Small pages	Large pages
Less internal fragmentation in the last page of a process.	Better disk transfer efficiency because secondary storage likes large blocks.
More pages per process -> larger page tables.	Fewer PTEs and better TLB reach for large contiguous regions.
More pages fit near recent references -> can improve locality and reduce faults.	Can weaken locality because each page contains memory farther away from the recent reference.
Can cause page tables to live in VM -> possible double page faults.	May waste memory if only a small part of a large page is used.

Modern issue: Object-oriented programs and multithreading can reduce locality by scattering references. One response is larger TLBs or multiple page sizes, but both have tradeoffs.

10. Paging performance: Effective Access Time

```
EAT = (1 - p) * ma + p * pft

p   = probability of page fault
ma  = memory access time
pft = page fault service time
```

- Lecture example: memory access time = 100 ns, page fault time = 25 ms = 25,000,000 ns.
- EAT = 100 + 24,999,900 * p approximately.
- If p = 0.001, EAT is about 25 microseconds, around 250x slower.
- To keep slowdown under 10%, page faults must be less than about 1 in 2,500,000 accesses.

Key aim: Minimize page faults. Even a tiny page fault probability can destroy performance because disk access is enormously slower than memory access.

11. Memory management policy decisions

Policy	Question it answers	Main options / notes
Fetch policy	When to bring a page into memory?	Demand paging vs prepaging. Prepaging brings extra pages but may waste I/O if unused.
Placement policy	Where to put the page/segment?	Important for pure segmentation; trivial for pure paging because any free

		frame works.
Replacement policy	What page should be removed when memory is full?	Random, OPT, FIFO, LRU, Clock, modified Clock.
Resident set management	How many frames should each process get?	Fixed or variable allocation; local or global scope.
Cleaning policy	When to write modified pages to disk?	Demand cleaning, precleaning, page buffering.
Load control	How many processes should be resident?	Too many -> thrashing; too few -> CPU idle / low utilization.

12. Page replacement algorithms

General goal: Replace the page least likely to be referenced in the near future. Since the future is unknown, algorithms predict future behavior from past behavior.

Algorithm	How it chooses victim	Strength	Weakness / exam trap
Random	Pick any page randomly.	Simple.	No intelligence; used mainly as a baseline.
Optimal (OPT)	Replace page whose next use is farthest in the future.	Fewest possible page faults.	Impossible in practice because future references are unknown; used for comparison.
FIFO	Replace the page that has been in memory the longest.	Very simple; circular queue.	Old page may still be heavily used. Lecture example says FIFO ignores that pages 2 and 5 are frequent.
LRU	Replace the page not referenced for the longest time in the past.	Approximates locality; close to optimal.	Expensive to implement exactly because it needs reference times or ordering.
Clock / NRU	Circular scan with use bit. Skip pages with use=1 and reset to 0. Replace first use=0.	Efficient approximation of LRU.	Not exact LRU; pointer position matters.
Modified Clock	Uses use bit + modified bit. Prefer clean and not recently used pages.	Avoids unnecessary disk writes.	Multiple passes; order of classes matters.

Lecture replacement example

- Reference string: 2 3 2 1 5 2 4 5 3 2 5 2.
- Fixed resident set size: 3 frames.
- The lecture marks faults after the initial frame allocation is filled: OPT produces 3, LRU produces 4, FIFO produces 6.
- Meaning: OPT is best but unrealizable; LRU usually performs close to OPT; FIFO is simple but can replace important pages.

Clock algorithm steps

1. Each frame has a use bit.
2. When a page is loaded or referenced, set use bit = 1.
3. On replacement, scan frames circularly using a pointer.
4. If current use bit is 1, set it to 0 and move on.
5. If current use bit is 0, replace that frame.
6. After replacement, pointer advances to the next frame.

Modified Clock classes

Class	Meaning	Replacement preference
u=0, m=0	Not recently used, clean	Best victim: no recent use and no disk write needed.
u=0, m=1	Not recently used, dirty	Good victim but requires write-back.
u=1, m=0	Recently used, clean	Avoid if possible because locality suggests it may be needed.
u=1, m=1	Recently used, dirty	Worst victim: recent and expensive to write.

13. Resident set management and replacement scope

- The OS must decide how many frames to give each process.
- Too few frames -> many page faults.
- Too many frames -> fewer processes fit in memory, possibly increasing CPU idle time or swapping.
- After a certain resident set size, extra frames may not reduce page faults because locality already fits.

Concept	Meaning
Fixed allocation	Process receives a fixed number of frames decided at load/creation time. On fault, replace one of its own pages.
Variable allocation	Number of frames changes over process lifetime based on behavior/page fault rate.
Equal allocation	Give each process the same number of frames.
Proportional allocation	Allocate frames according to process size, or priority in a variant.
Local replacement	Victim page must come from the faulting process resident set.
Global replacement	Victim page can come from any unlocked frame in memory.

Global vs local scope

- **Local replacement:** More predictable per process. A bad process cannot steal frames from everyone, but it may be stuck with too few frames.
- **Global replacement:** Often easier to implement and allows dynamic sharing of memory, but one process can reduce another process resident set.
- **Frame locking:** Some frames cannot be replaced, such as kernel pages, control structures, and I/O buffers.

Fixed allocation, local scope

- Decide each process allocation ahead of time based on process type, request, or guidance.
- If too small: high page fault rate.
- If too large: too few programs in memory, more idle CPU or more swapping.

Variable allocation, global scope

- Easy and used by many operating systems.
- A page fault can add a free frame to the faulting process resident set.
- If no free frame exists, replacement can select from all unlocked frames.
- Hard part: the victim may come from the wrong process, hurting its performance.

Variable allocation, local scope

- Start with an initial resident set based on process type or other criteria.
- On page fault, replace from the same process resident set.
- Periodically reevaluate and increase/decrease the allocation based on future demand estimates.
- More complex and higher overhead, but can give better performance.

14. Working set and Page-Fault Frequency

- **Working set W(t, delta):** The set of pages referenced during the last delta time units at time t.
- It approximates locality. If the working set is in memory, page faults should be low.
- If delta is too small, it misses part of the locality. If too large, it may include several localities.
- Processes have stable phases and transient phases. During transients, the working set changes rapidly.

Working set strategy: Monitor each process, remove pages not in its working set, and allow a process to execute only if its working set is resident. This is appealing but hard to implement exactly because it needs accurate tracking of every page reference.

Page-Fault Frequency (PFF) scheme

- Establish an acceptable page fault rate.
- If fault rate is too low: the process may lose frames.
- If fault rate is too high: the process should gain frames.
- Approximate algorithm: use a use bit for each page; on each page fault, measure time since last fault T.
- If T < F1, add page to resident set. If T > F2, discard pages with use bit = 0 and reset the remaining use bits.
- Weakness: transient phases may cause resident set swelling before old locality pages are removed.

15. Cleaning policy, page buffering, and load control

Concept	Meaning	Exam note
Demand cleaning	Write a modified page only when it is selected for replacement.	May delay replacement because dirty page must be written now.
Precleaning	Write modified pages in batches before they are selected.	Can waste I/O if pages are modified again soon.
Page buffering	Keep replaced pages in free/modified lists as a cache.	Useful because a recently replaced page may be reclaimed cheaply.
Free page list	Frames available for incoming pages.	OS tries to keep some frames free.
Modified page list	Dirty pages waiting to be written.	Can be written out in clusters to reduce I/O.

Load control

- Load control determines the degree of multiprogramming: how many processes are resident in memory.
- Increasing multiprogramming initially increases CPU utilization.
- Too many resident processes -> each gets too few frames -> page fault rate rises -> thrashing.
- Too few resident processes -> higher chance all are blocked -> CPU idle time rises.

If the OS must reduce multiprogramming, which process to suspend?

Candidate	Reason
Lowest priority process	Matches scheduling policy, but not necessarily performance optimal.
Faulting process	It is already blocked and likely lacks its working set.
Last process activated	Least likely to have its working set resident.
Smallest resident set	Cheapest to reload later, but may penalize good locality.
Largest process	Frees most frames quickly.
Largest remaining execution window	May reduce future memory pressure.

16. Program structure and locality example

- Program structure can massively affect page faults.

- If a 128 x 128 array is stored by rows and each row fits in one page, row-major traversal causes far fewer page faults than column-major traversal.

```
Bad locality example (column-wise for row-stored array):
for (j = 0; j < 128; j++)
  for (i = 0; i < 128; i++)
    data[i,j] = 0;
Lecture result: 16,384 page faults.

Good locality example (row-wise for row-stored array):
for (i = 0; i < 128; i++)
  for (j = 0; j < 128; j++)
    data[i,j] = 0;
Lecture result: 128 page faults.
```

Exam trap: The difference is not the number of assignments. Both programs write the same array. The difference is memory access order and locality.

17. Likely exam questions and direct answers

- **Define virtual memory.** A technique that separates logical memory from physical memory so only part of a process must be in RAM, allowing processes larger than physical memory.
- **Why does virtual memory work?** Because of locality: processes usually reference a small cluster of pages over a short time.
- **What is demand paging?** Loading a page only when it is referenced during execution.
- **What happens on a page fault?** Trap to OS, validate reference, find free frame, read page from disk, update page table, restart instruction.
- **What is the resident set?** The portion of a process currently in main memory.
- **What is thrashing?** Excessive paging where CPU spends most time swapping pages, not executing instructions.
- **TLB miss vs page fault?** TLB miss means translation is not cached. Page fault means page is not resident/valid in memory. A TLB miss may still find the page in the page table.
- **Why is dirty bit useful?** A clean victim can be discarded without writing to disk; a dirty victim must be written back.
- **Why is OPT not practical?** It requires perfect knowledge of future page references.
- **Why can FIFO perform poorly?** The oldest page may still be frequently used.
- **Why is exact LRU expensive?** It needs to track the last reference time/order for every page.
- **What does Clock approximate?** LRU, using a use/reference bit and circular scan.
- **What is working set?** Pages referenced within the last delta time units; an approximation of current locality.
- **What is page buffering?** Keeping replaced pages in free/modified lists so they can be reclaimed or written in batches.
- **What is load control?** Choosing the number of resident processes in memory to avoid too few processes or too much thrashing.

18. Final memorization sheet

Term	Say this in the exam
VM	Logical memory separated from physical memory; only needed pieces in RAM.
Demand paging	Bring page only on reference.
Page fault	Missing page -> trap -> OS loads page -> restart instruction.
Resident set	Process pages currently in RAM.
Locality	References cluster; VM depends on this.

Thrashing	Too many page faults; CPU mostly swaps.
PTE bits	Frame, present, modified/dirty, accessed/use, protection.
TLB	Cache of PTEs; TLB miss is not necessarily a page fault.
OPT	Best but impossible.
LRU	Good but expensive.
FIFO	Simple but can be bad.
Clock	Efficient LRU approximation.
Working set	Pages used in recent time window.
PFF	High fault rate -> give frames; low fault rate -> remove frames.
Load control	Too many processes -> thrashing; too few -> idle CPU.